

Mobile App Ecosystem Dynamics and Direct Bank Marketing Pipelines

Exploratory data analysis, regression and classification on four real-world datasets

Group 10 member	Register number
Sanskriti Rastogi	RA2411003012439
Sahil Raj	RA2411003012492
Yash Santosh Lohare	RA2411003012493
Om Pathania	RA2411003012495

Guide: Dr. Arnab Maity

Review: PBL Review 1, 28 September 2026

Tools: Python 3.13, pandas, scikit-learn, XGBoost, imbalanced-learn, matplotlib, seaborn

Contents

	Abstract
1	Introduction
2	Datasets
3	Methodology
4	Exploratory Data Analysis
	4.1 Google Play Store apps · 4.2 Video game sales
5	Regression: Combined Cycle Power Plant
6	Classification: Bank Marketing
7	Comparative Discussion
8	Conclusion, Limitations and Future Work
	References
	Appendix: Reproducibility

Abstract

This project studies the digital economy from two sides: the markets for mobile apps and video games, and a bank's direct-marketing campaign. It builds complete machine-learning pipelines around four public datasets. Exploratory analysis of 10,841 Google Play Store listings shows that installs concentrate in the Game and Communication categories, that 92% of apps are free, and that recently updated apps are rated higher. Analysis of 16,323 video games shows a hit-driven market that peaked in 2008, with North America providing 49% of sales and Japan behaving as a distinct market. For regression, eleven models were compared on the Combined Cycle Power Plant dataset. A tuned XGBoost model predicts hourly energy output with an RMSE of 3.17 MW ($R^2 = 0.966$), 31% lower error than linear regression. For classification, ten approaches were compared on the imbalanced Bank Marketing dataset (11% positives) after removing the leaking `duration` feature. A class-weighted XGBoost model with a tuned decision threshold achieves $F1 = 0.531$ and $ROC-AUC = 0.812$. Calling the top 20% of clients it ranks reaches 67% of all subscribers, a $3.3\times$ lift over random calling.

1 Introduction

1.1 Problem statement

Digital businesses make decisions on large, messy data. App developers and game publishers need to know which products succeed and where demand comes from. Banks running outbound call campaigns need to know which customers are worth contacting, because most calls fail: in our data, 89% of contacted clients decline. The problem statement assigned to Group 10, “*Mobile App Ecosystem Dynamics and Direct Bank Marketing Pipelines*”, asks us to (i) understand the dynamics of two digital markets through exploratory analysis, and (ii) build end-to-end predictive pipelines for a continuous and a binary business outcome.

1.2 Objectives

1. **Mobile app ecosystem (EDA 1):** identify what is associated with app ratings and installs: category, price model, size, update frequency.
2. **Video game industry (EDA 2):** describe the industry's sales lifecycle from 1980 to 2016, and which genres, platforms, publishers and regions dominate.
3. **Regression:** predict a power plant's net hourly electrical output (MW) from ambient conditions and compare linear and non-linear regressors.
4. **Classification:** predict before calling whether a bank client will subscribe to a term deposit; handle class imbalance and choose metrics that reflect the business goal.

1.3 Mapping to the evaluation rubric

Rubric criterion (4 marks each)	Where it is addressed
Problem Identification & Dataset Selection	Sections 1 and 2
Exploratory Data Analysis & Insights	Section 4 (plus modelling EDA in 5.1 and 6.1)
Regression and Classification Implementation	Section 3, 5.2 and 6.2
Comparative Performance Analysis	Sections 5.3, 6.3 and 7
Result Interpretation & Presentation	Sections 5.4, 6.4 and 8

2 Datasets

We use the four datasets assigned to Group 10. Each was chosen because its structure suits a particular task.

Table 1. Datasets used in this project.

Task	Dataset	Source	Rows × cols	Target
EDA 1	Google Play Store Apps	Kaggle (lava18)	10,841 × 13	– (exploratory)
EDA 2	Video Game Sales	Kaggle (gregorut)	16,598 × 11	– (exploratory)
Regression	Combined Cycle Power Plant	UCI ML Repository #294	9,568 × 5	PE, net energy output (MW)
Classification	Bank Marketing (bank-additional-full)	UCI ML Repository #222	41,188 × 21	y, subscribed (yes/no)

Google Play Store Apps

A 2018 scrape of Play Store listings: app name, category, rating, number of reviews, size, installs bucket, free/paid type, price, content rating, genres, last-update date and version fields. It is deliberately messy: numeric fields are stored as text ("19M", "10,000+", "\$4.99"), there are duplicates and a corrupted row, and 13.6% of raw ratings are missing. That makes it a realistic data-cleaning exercise.

Video Game Sales

Every game that sold more than 100,000 copies (scraped from VGChartz), with platform, release year, genre, publisher and sales in millions of units for North America, Europe, Japan, the rest of the world, and globally. It covers 37 years, which suits lifecycle, concentration and regional-preference analysis.

Combined Cycle Power Plant (CCPP)

9,568 hourly averages collected over six years (2006–2011) from a plant running at full load. The four inputs are ambient temperature AT (°C), exhaust vacuum V (cm Hg), ambient pressure AP (mbar) and relative humidity RH (%). The target is the net hourly electrical energy output PE (MW). The data is clean and physically meaningful, which makes it a good benchmark for comparing regression algorithms.

Bank Marketing

Records of phone-based marketing campaigns run by a Portuguese bank between May 2008 and November 2010 (Moro et al., 2014). The 20 inputs describe the client (age, job, marital status, education, credit default, housing and personal loans), the current campaign contact (channel, month, weekday, call duration, number of contacts), previous campaigns (days since last contact, previous contacts and outcome), and five quarterly macro-economic indicators. The target *y* is whether the client subscribed to a term deposit. Only 11.3% did, which makes this a realistic imbalanced classification problem.

3 Methodology

3.1 Pipeline

All four tasks follow the same pipeline: **audit** → **clean** → **explore** → **engineer features** → **model** → **compare** → **interpret**. The work lives in four Jupyter notebooks (Appendix). Every figure and number in this report is generated by those notebooks and saved to `code/outputs/`, so the report can be reproduced exactly.

3.2 Preventing data leakage

- **Train/test separation:** an 80/20 split is made first (stratified for classification). The test set is used once, for the final comparison.
- **Pipelines:** scaling, one-hot encoding and SMOTE oversampling are wrapped in scikit-learn / imbalanced-learn `Pipeline` objects. They are therefore re-fitted inside every cross-validation fold and never see validation or test rows.
- **Feature leakage:** the Bank Marketing field `duration` (call length) is only known after the call ends and nearly determines the outcome, so it is excluded from the models (Section 6.1).
- **Threshold selection:** the classification threshold is chosen on out-of-fold predictions from the training set, not on the test set.

3.3 Models

Regression: Linear Regression, Ridge (L2), Lasso (L1), Polynomial Regression (degree 3), k-Nearest Neighbours, Support Vector Regression (RBF kernel), Decision Tree, Random Forest, Gradient Boosting and XGBoost. The best family (XGBoost) was tuned with `GridSearchCV` over 16 settings.

Classification: a majority-class baseline, Logistic Regression (plain, class-weighted and with SMOTE), Gaussian Naive Bayes, k-NN, Decision Tree, Random Forest, Gradient Boosting and XGBoost.

3.4 Evaluation metrics

- **Regression:** MAE (average absolute error in MW), RMSE (penalises large errors; the primary metric) and R^2 (share of variance explained), all measured with 5-fold cross-validation and on the held-out test set.
- **Classification:** accuracy is reported but not used for ranking, because the majority-class baseline already scores 88.7%. We use **precision** (share of predicted subscribers who subscribe), **recall** (share of subscribers found), **F1** (their harmonic mean), **ROC-AUC** (ranking quality, threshold-free) and **PR-AUC** (ranking quality focused on the rare positive class). Stratified 5-fold CV is used on the training set.

4 Exploratory Data Analysis

4.1 Google Play Store apps

Data quality and cleaning

10,841 →
9,659

rows after cleaning

1,182

duplicate / corrupt rows
removed

15.1%

apps without a rating
(after de-duplication)

92%

of unrated apps have $\leq$
1,000 installs

- **Corrupted row:** row 10472 has its columns shifted by one (Category “1.9”, Rating 19), so it was dropped.
- **Duplicates:** 483 exact duplicate rows, plus repeated scrapes of the same app with different review counts. We kept one row per app, the one with the most reviews (the latest scrape).
- **Type conversion:** Reviews to integer; Size (“19M”, “91k”) to megabytes, with 1,695 “Varies with device” values set to missing; Installs (“10,000+”) to integer; Price (“\$4.99”) to float; Last Updated to a date.
- **Missing ratings:** 92% of unrated apps have 1,000 installs or fewer, so ratings are missing because apps are too small to be rated, not at random. Imputing a mean would invent scores, so they are left as missing and excluded only from rating analyses.
- **Engineered features:** days since last update, primary genre, $\log_{10}(\text{installs})$, $\log_{10}(\text{reviews})$.

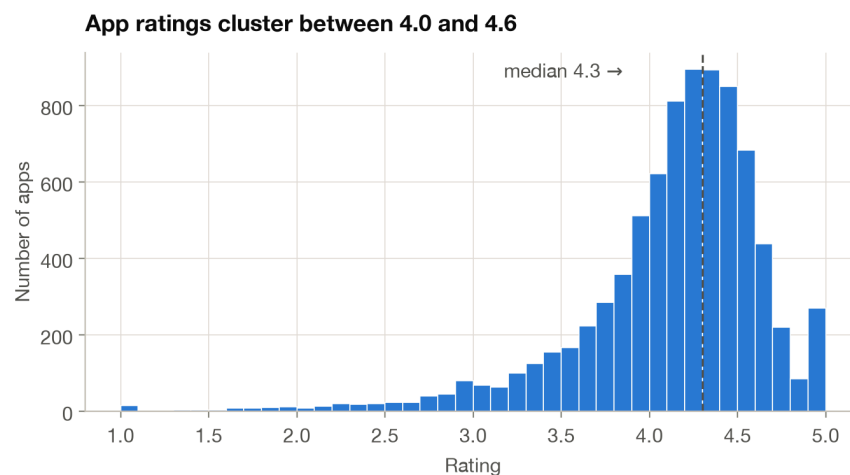


Figure 1. Distribution of app ratings. Median 4.3; 77% of rated apps score 4.0 or higher.

Findings

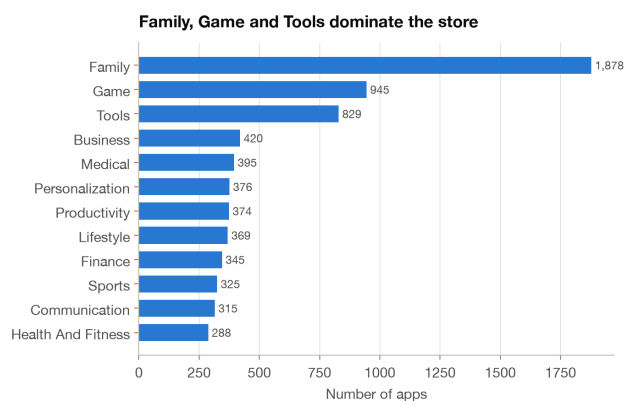


Figure 2. Number of apps per category (top 12).

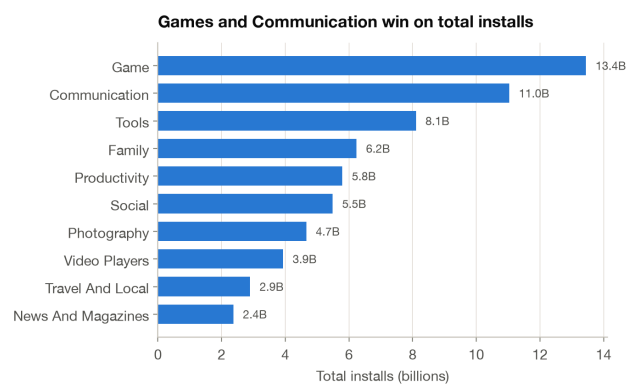


Figure 3. Total installs per category (top 10).

Supply does not equal demand. Family is the largest category by number of apps (1,878), but Game (13.4 billion installs) and Communication (11.0 billion) receive the most installs. Crowded categories are not necessarily where users are.

Free vs Paid apps

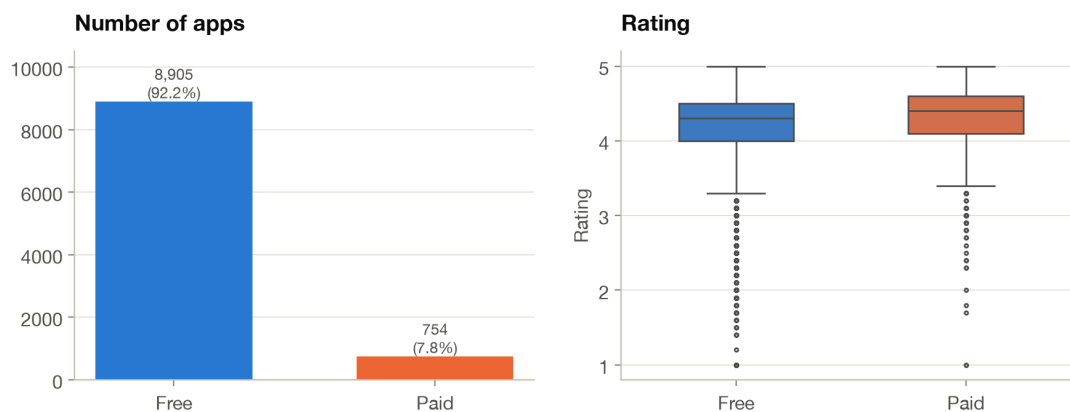


Figure 4. Free vs paid apps: count (left) and rating distribution (right).

Free apps dominate. 92.2% of apps are free. The median free app has 100,000 installs against 1,000 for a paid app (median price \$2.99). Paid apps are rated slightly higher (4.26 vs 4.17), which is consistent with a smaller, more committed audience.

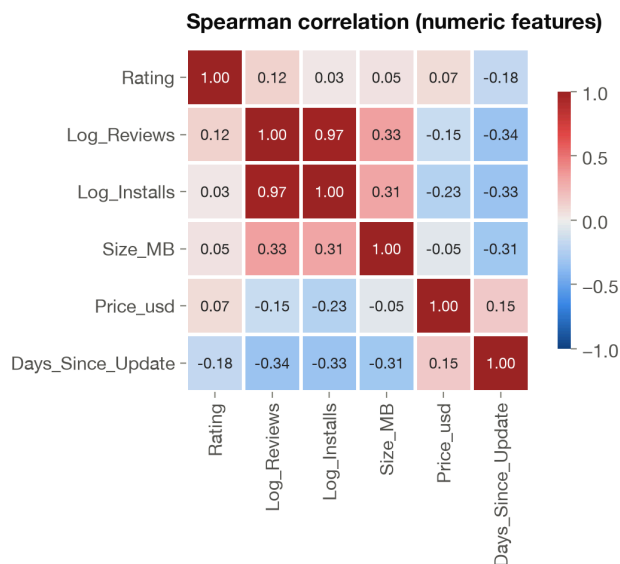


Figure 5. Spearman correlations among numeric features.

Popularity measures move together; quality does not. Reviews and installs are almost perfectly rank-correlated ($\rho = 0.97$), so review count is a reliable proxy for popularity. Rating is only weakly related to reviews ($\rho = 0.12$) and size ($\rho = 0.05$). Its strongest numeric association is with days since the last update ($\rho = -0.18$).

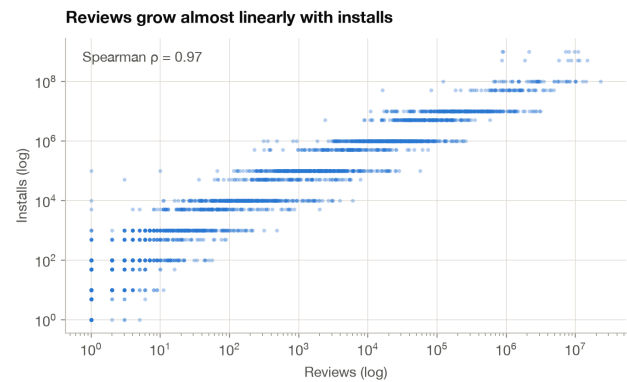


Figure 6. Reviews vs installs (log-log, 3,000-app sample).

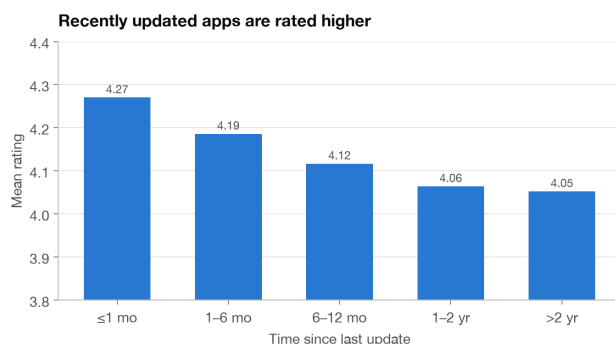


Figure 7. Mean rating by time since last update.

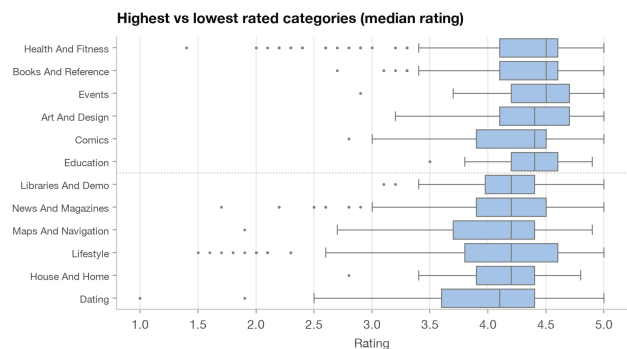


Figure 8. Highest- and lowest-rated categories by median rating.

Maintenance pays. Apps updated within the last month average 4.27 stars, against 4.05 for apps untouched for more than two years. Health & Fitness, Books and Events have the highest median ratings; Dating has the lowest.

4.2 Video game sales

Data quality and cleaning

- No duplicate rows. Regional sales add up to global sales, apart from rounding.
- 271 games (1.6%) with no release year were dropped. Years after 2016 contain only 4 games (the data was scraped during 2016), so the analysis covers 1980–2016: 16,323 games.
- 58 missing publishers were labelled “Unknown”. Extreme sales values are genuine hits, not errors, so they were kept and plotted on log scales.

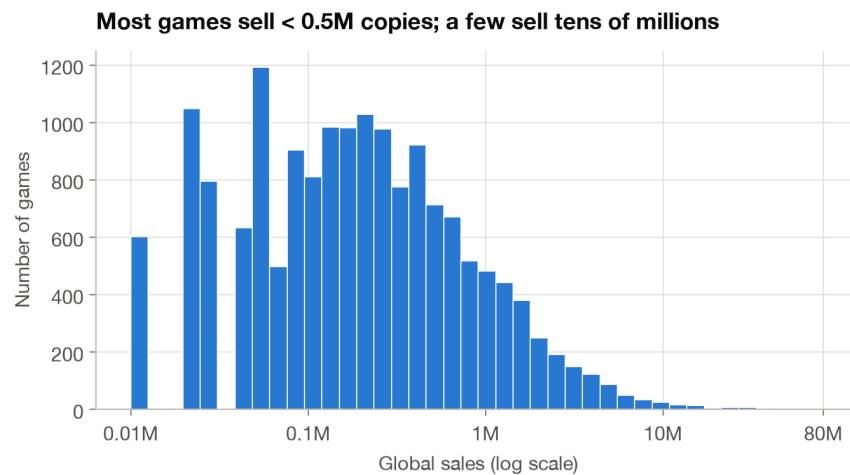


Figure 9. Distribution of global sales per game (log scale). Skewness = 17.3.

A hit-driven market. The median game sells 0.17 million copies, while Wii Sports sold 82.7 million. The top 1% of games account for 21% of all sales.

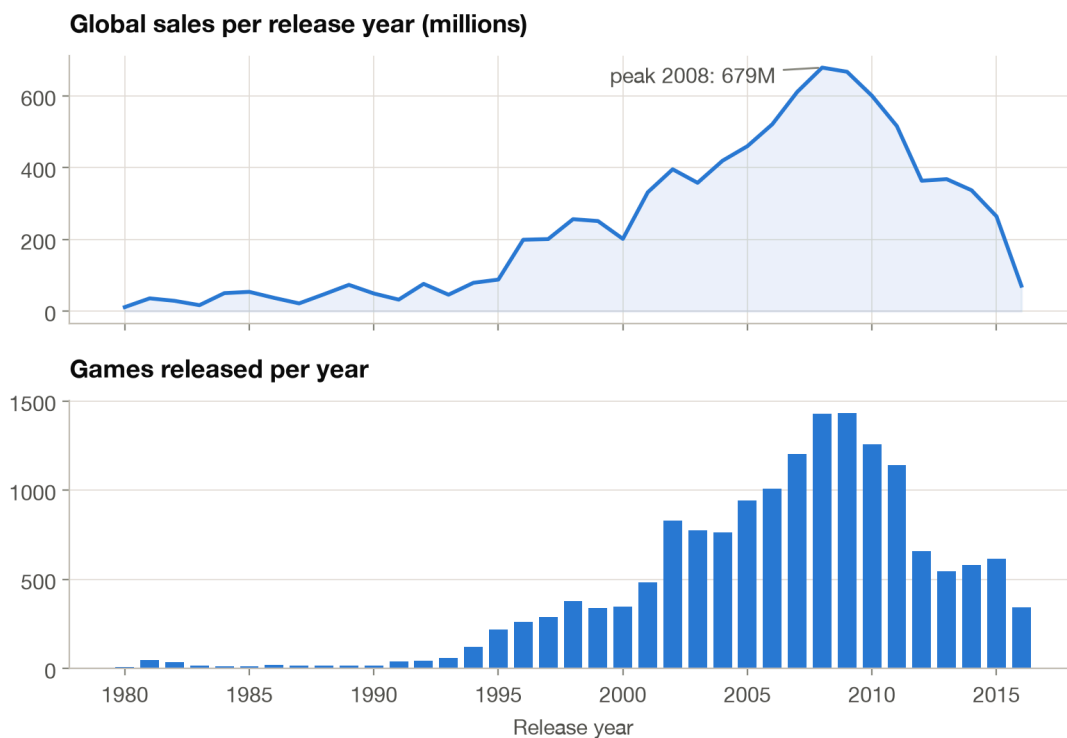


Figure 10. Industry lifecycle: total global sales and number of games released per year.

Industry lifecycle. Sales grew steadily from the mid-1990s to a peak in 2008 (679 million units from games released that year); releases peaked in 2009. Both then fell sharply. Part of the decline is real, as players moved to mobile and digital distribution. Part is an artefact of the data, which counts boxed retail units and under-represents recent titles still on sale when it was scraped.

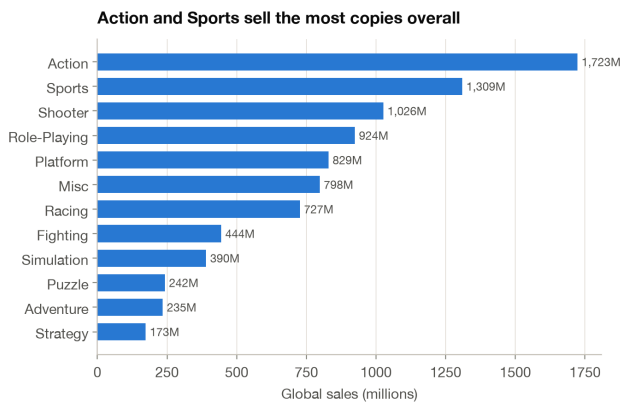


Figure 11. Total global sales by genre.

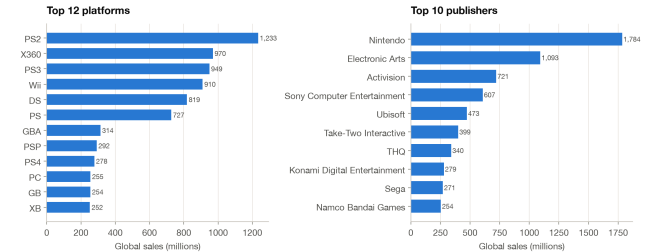


Figure 12. Top platforms and publishers by global sales.

Genres, platforms, publishers. Action sells the most in total (1,723M), but Platform games sell the most per title (0.95M on average): fewer, bigger hits. The PS2 is the best-selling platform (1,233M). The market is concentrated: of 576 publishers, the top five (Nintendo, Electronic Arts, Activision, Sony, Ubisoft) hold 53% of sales.

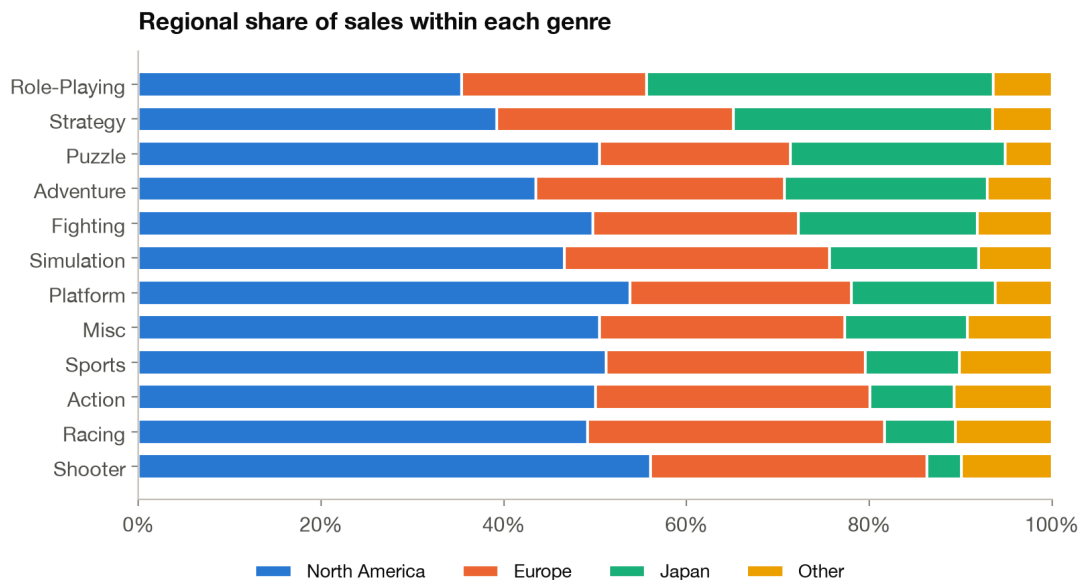


Figure 13. Share of each genre's sales by region.

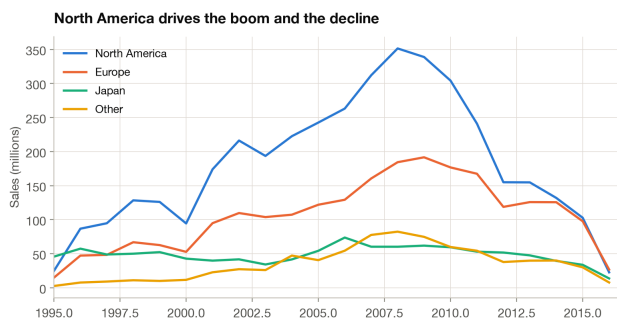


Figure 14. Regional sales per year, 1995–2016.

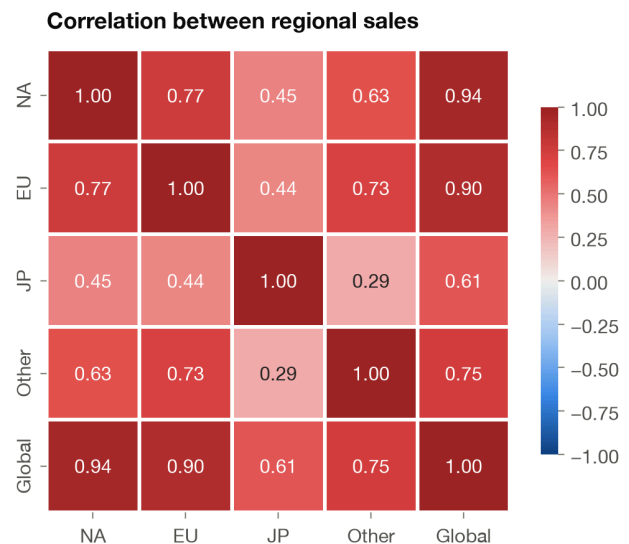


Figure 15. Correlation between regional sales.

Regional taste. North America provides 49.1% of all sales, Europe 27.3%, Japan 14.6% and the rest of the world 8.9%. North American sales correlate 0.94 with global sales, but Japan only 0.61: Japan is a distinct market. Role-Playing is Japan's top genre, and 38% of all RPG sales come from Japan. Shooters are 56% North American. Publishers should localise their genre mix by region rather than follow one global strategy.

5 Regression: Combined Cycle Power Plant

5.1 Data preparation and modelling EDA

The Excel file contains five sheets holding the same rows in different orders, so we used Sheet 1. There are no missing values; 41 duplicate rows were removed, leaving 9,527 rows. IQR outliers were inspected and kept, because all values are physically plausible sensor readings.

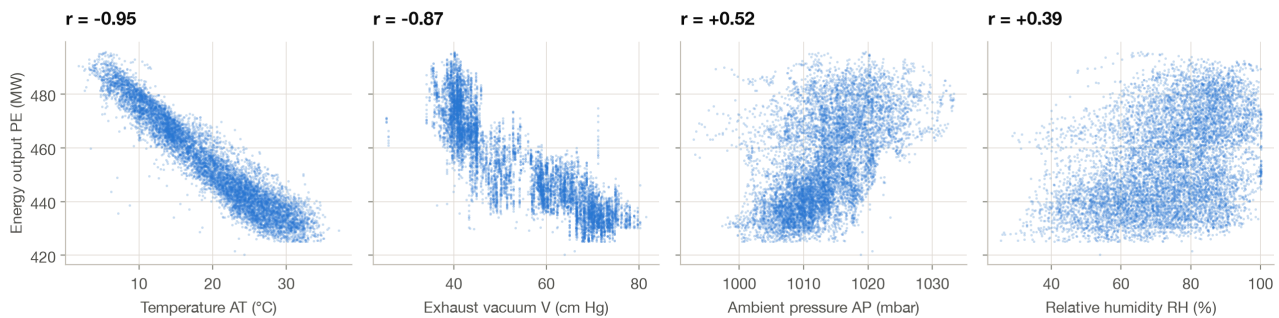
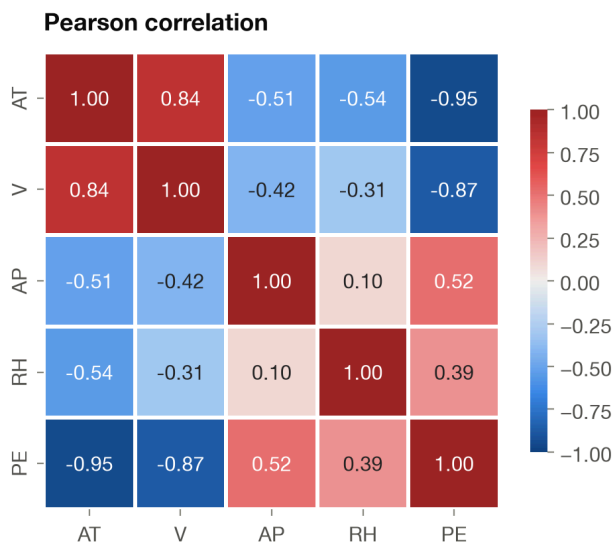


Figure 16. Each input feature against energy output PE, with Pearson correlation r .



Observations. Temperature ($r = -0.95$) and exhaust vacuum ($r = -0.87$) are strong, nearly linear negative drivers. Hotter, less dense air reduces gas-turbine output. Pressure ($+0.52$) and humidity ($+0.39$) are weaker.

AT and V are correlated with each other ($r = 0.84$; VIF AT = 6.0, V = 3.9). This is moderate multicollinearity: acceptable for prediction, but individual linear coefficients must be interpreted with care. The AT–PE relationship curves slightly, which suggests that non-linear models should help.

Figure 17. Pearson correlation matrix.

5.2 Implementation

- Train/test split: 80/20 (7,621 / 1,906 rows), `random_state = 42`.
- Scale-sensitive models (linear family, polynomial, k-NN, SVR) use `StandardScaler` inside a `Pipeline`; tree ensembles use raw features.
- 5-fold cross-validation (shuffled) on the training set for every model; test-set evaluation once at the end.
- Hyper-parameter tuning: `GridSearchCV` for XGBoost over $n_estimators \in \{600, 1200\}$, $learning_rate \in \{0.03, 0.05\}$, $max_depth \in \{5, 7\}$, $colsample_bytree \in \{0.8, 1.0\}$. Best: $colsample_bytree = 1.0$, $learning_rate = 0.03$, $max_depth = 7$, $n_estimators = 1200$.

5.3 Comparative performance

Table 2. Regression results, sorted by test RMSE. CV = 5-fold cross-validation on the training set.

Model	CV R ²	CV std	Train R ²	MAE (MW)	RMSE (MW)	Test R ²
XGBoost (tuned)	0.9670	0.0026	0.9958	2.166	3.171	0.9658
XGBoost	0.9656	0.0026	0.9885	2.287	3.282	0.9633
Gradient Boosting	0.9623	0.0030	0.9801	2.441	3.404	0.9606
Random Forest	0.9598	0.0033	0.9948	2.409	3.434	0.9599
KNN (k=7)	0.9510	0.0021	1.0000	2.651	3.750	0.9521
Decision Tree	0.9417	0.0037	0.9682	2.933	4.075	0.9435
SVR (RBF)	0.9444	0.0035	0.9469	3.006	4.077	0.9434
Polynomial (deg 3)	0.9404	0.0034	0.9411	3.269	4.234	0.9390
Lasso ($\alpha=0.01$)	0.9283	0.0034	0.9284	3.621	4.587	0.9284
Ridge ($\alpha=1$)	0.9283	0.0034	0.9284	3.622	4.587	0.9284
Linear Regression	0.9283	0.0034	0.9284	3.621	4.587	0.9284

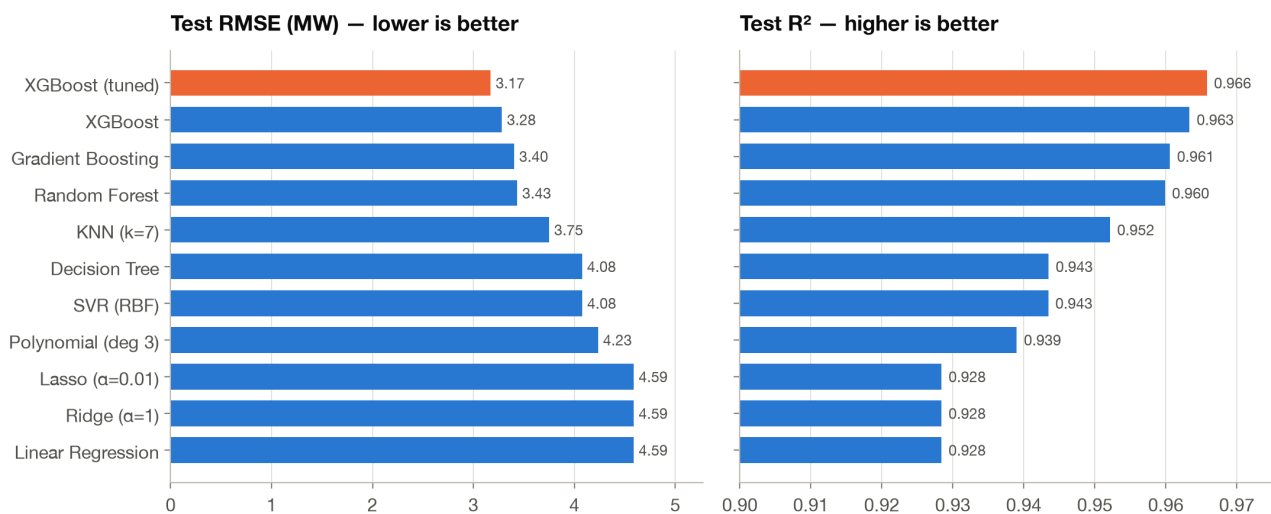


Figure 18. Test RMSE and R² for all regression models; the best model is highlighted.

- **Tuned XGBoost is best:** RMSE 3.17 MW, MAE 2.17 MW, R² 0.966. That is 31% lower RMSE than Linear Regression (4.59 MW). Untuned XGBoost, Gradient Boosting and Random Forest follow closely.
- **Linear models already explain 93% of variance,** because AT and V act almost linearly. Ridge and Lasso match plain Linear Regression: with four features and about 7,600 training rows there is no overfitting for regularisation to correct.
- **Model families rank as expected:** ensembles > k-NN > single tree $\approx$ SVR > polynomial > linear. Capturing curvature and the AT $\times$ V interaction is what produces the gains.
- **Stability:** the CV R² standard deviation is about 0.003 for every model, and CV R² closely matches test R², so the ranking is not an artefact of one split. Tree models reach train R² above 0.99 (some

memorisation), but cross-validation confirms this does not hurt their test error.

5.4 Interpretation of the best model

XGBoost (tuned): test-set diagnostics

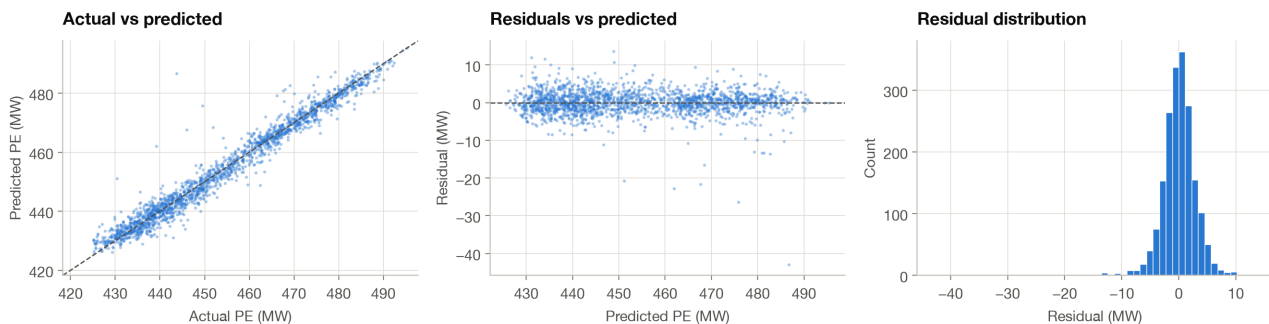


Figure 19. Test-set diagnostics for tuned XGBoost: actual vs predicted, residuals vs predicted, residual distribution.

Predictions lie tightly along the diagonal. Residuals are centred at zero, roughly symmetric, and show no trend across the output range, so the model has no systematic bias. 92% of hourly predictions fall within ± 5 MW of the true value, where output ranges from 420 to 496 MW.

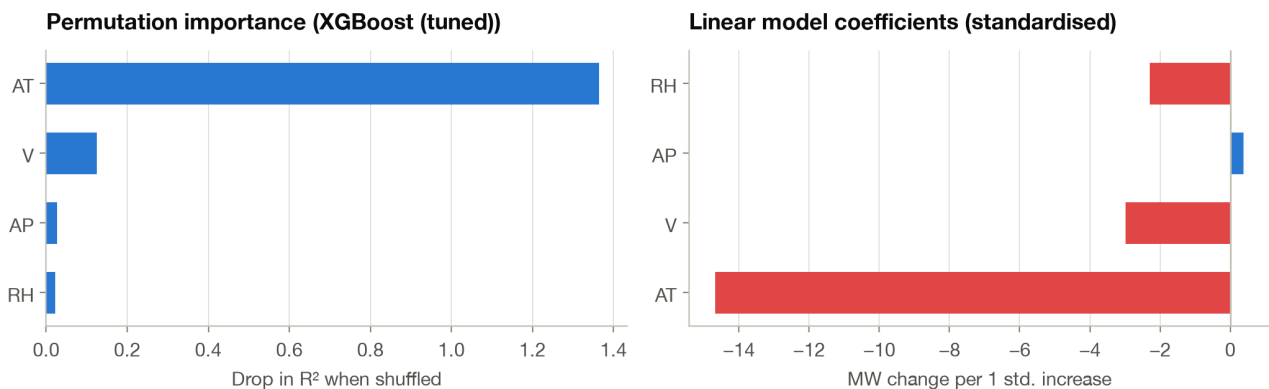


Figure 20. Permutation importance for tuned XGBoost (left) and standardised linear coefficients (right).

Temperature dominates. Randomly shuffling AT lowers R² by 1.37, which destroys the model; V is a distant second (0.13), and AP and RH add small corrections. In the linear model, a one-standard-deviation rise in temperature (about 7.5 °C) lowers output by 14.7 MW. **Practical use:** feeding a weather forecast into the model gives an hour-ahead output forecast that operators can use for grid dispatch and energy trading.

6 Classification: Bank Marketing

6.1 Data preparation and modelling EDA

The file is semicolon-separated. It has no NaN values, but `unknown` is used as a placeholder (e.g. 8,597 in `default`, 1,731 in `education`). We removed 12 duplicate rows. `unknown` was kept as its own category, because it carries signal: clients with unknown default status subscribe less often.

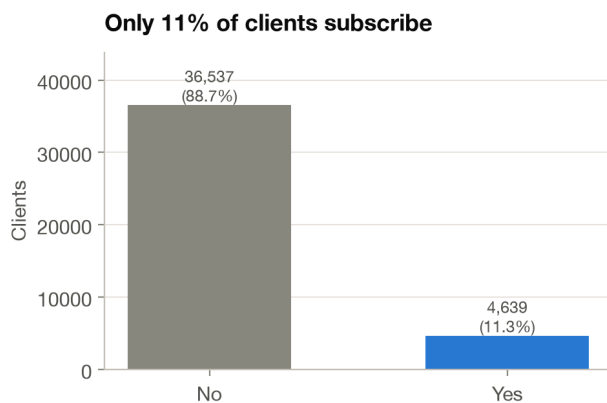


Figure 21. Class balance of the target.

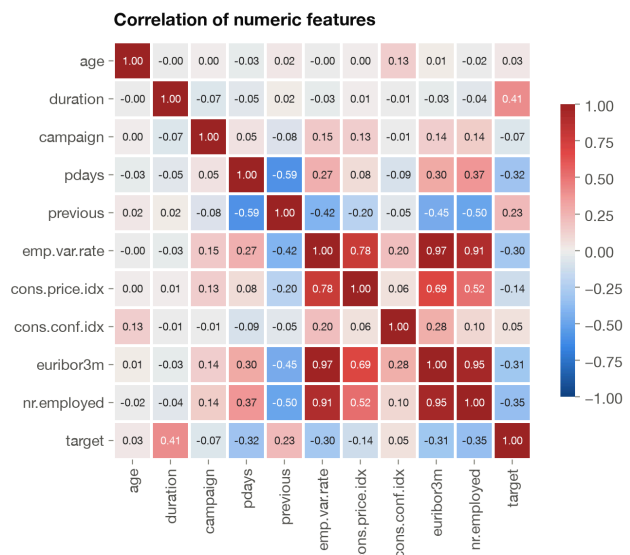


Figure 22. Correlation of numeric features with each other and the target.

Target leakage: duration removed. Call duration has the strongest correlation with the target ($r = 0.41$), but it is only known after the call has ended, and a very short call almost always means “no”. A model using it cannot decide whom to call, and its accuracy would be misleadingly high. We therefore excluded it, following the dataset authors' recommendation for realistic models.

Subscription rate by segment (dashed line = overall rate, 11%)

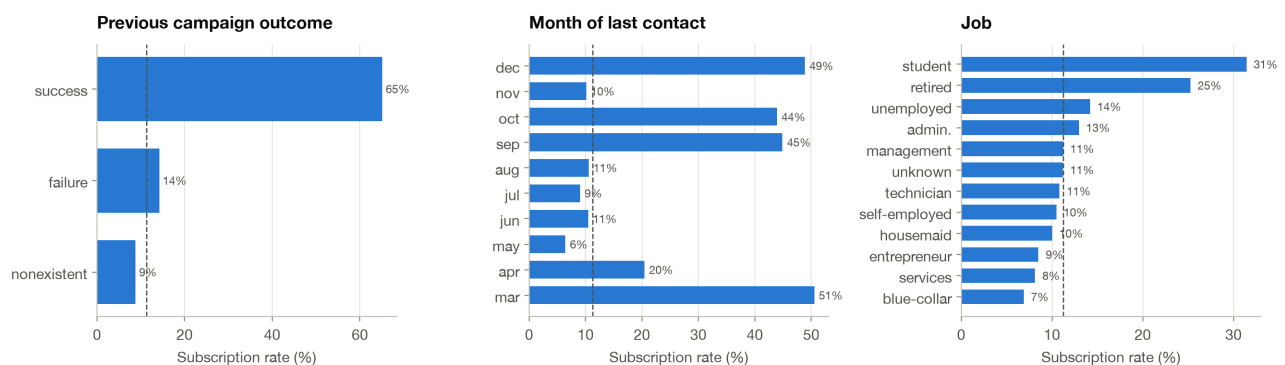


Figure 23. Subscription rate by previous-campaign outcome, month and job (dashed line: overall 11%).

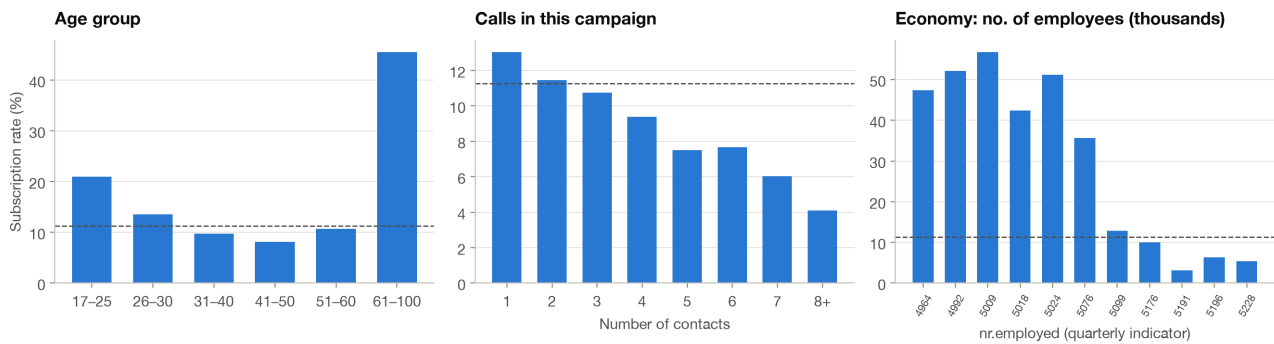


Figure 24. Subscription rate by age group, number of calls this campaign, and employment level.

- **Past success is the strongest single signal:** clients whose previous campaign succeeded subscribe 65% of the time, against 11% overall.
- **Timing:** March, September, October and December convert at 44–51%, but together make up under 5% of calls; May has the most calls (13,767) and the lowest rate (6.4%).
- **Diminishing returns:** the success rate falls with each additional call to the same client in a campaign.
- **Segments:** students (31%) and retirees (25%) are the most receptive; blue-collar workers (7%) the least.
- **Economy:** subscription rates are far higher when employment and Euribor rates are low. The five macro indicators are strongly inter-correlated (up to 0.97).

6.2 Implementation

- **Feature engineering:** dropped `duration`; replaced `pdays` (999 = never contacted, 96% of rows) with a binary `prev_contacted` flag. That leaves 19 input features.
- **Preprocessing:** a `ColumnTransformer` one-hot encodes the 10 categorical features and standardises the 9 numeric features, inside each model's pipeline.
- **Split:** stratified 80/20 (32,940 / 8,236 rows); both sets keep the 11.3% positive rate.
- **Imbalance handling, compared:** (a) no handling; (b) class weights (`class_weight="balanced"`, `XGBoost scale_pos_weight = 7.9`); (c) SMOTE applied inside CV folds only; (d) decision-threshold tuning.
- **Threshold tuning:** for the best model by CV ROC-AUC (XGBoost), 5-fold out-of-fold probabilities on the training set were used to find the F1-maximising threshold, 0.65. That threshold was then applied unchanged to the test set.

6.3 Comparative performance

Table 3. Classification results on the test set (n = 8,236), sorted by F1. CV ROC-AUC = stratified 5-fold on the training set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	CV ROC-AUC
XGBoost (weighted) @ thr 0.65	0.881	0.479	0.595	0.531	0.812	0.484	0.801
Random Forest (balanced)	0.873	0.454	0.617	0.523	0.807	0.484	0.794
XGBoost (weighted)	0.845	0.388	0.654	0.487	0.812	0.484	0.801
Decision Tree (balanced)	0.847	0.390	0.637	0.484	0.801	0.460	0.781
Logistic Reg. (balanced)	0.831	0.360	0.645	0.462	0.800	0.446	0.791
Logistic Reg. + SMOTE	0.822	0.347	0.654	0.453	0.799	0.445	0.787
Naive Bayes	0.808	0.320	0.624	0.423	0.776	0.359	0.766
KNN (k=15)	0.898	0.612	0.263	0.368	0.773	0.419	0.759
Gradient Boosting	0.899	0.645	0.233	0.342	0.811	0.480	0.797
Logistic Regression	0.898	0.652	0.212	0.320	0.801	0.453	0.790
Baseline (always 'no')	0.887	0.000	0.000	0.000	0.500	0.113	0.500

Test-set comparison (higher is better)

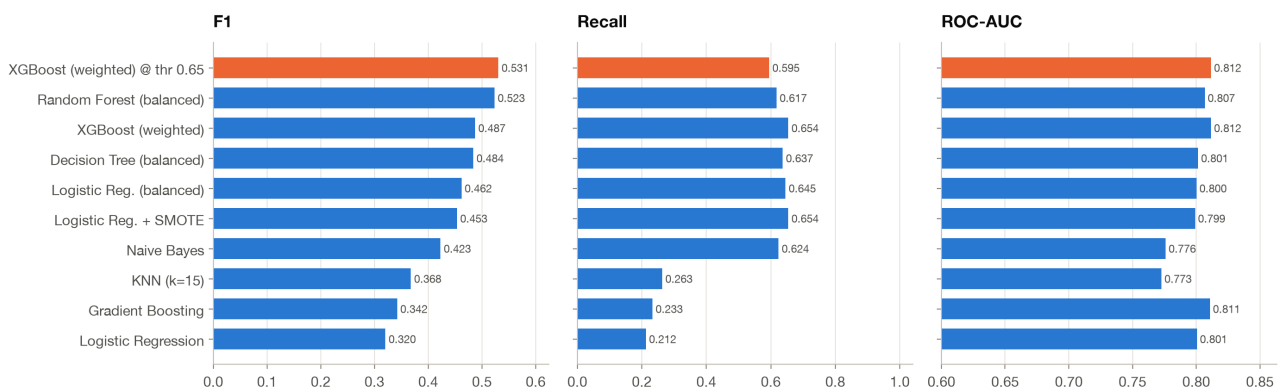


Figure 25. Test F1, recall and ROC-AUC for all models (baseline omitted).

- **The accuracy trap:** the “always no” baseline scores 88.7% accuracy with zero recall. Plain Logistic Regression and Gradient Boosting reach about 90% accuracy but find only 21–23% of subscribers.
- **Imbalance handling roughly triples recall** (to 0.60–0.65) at almost the same ROC-AUC. Class weights and SMOTE perform about equally; class weights are simpler and faster.
- **Best model:** class-weighted XGBoost has the best CV ROC-AUC (0.801) and test ROC-AUC (0.812). With the tuned threshold it also has the best F1 (0.531; precision 0.479, recall 0.595). Balanced Random Forest is a close second (F1 0.523).
- **Ceiling:** all good models cluster at ROC-AUC 0.80–0.81. The information available before a call is limited; models that include `duration` commonly report AUC above 0.9, which shows how much leakage would inflate the results.

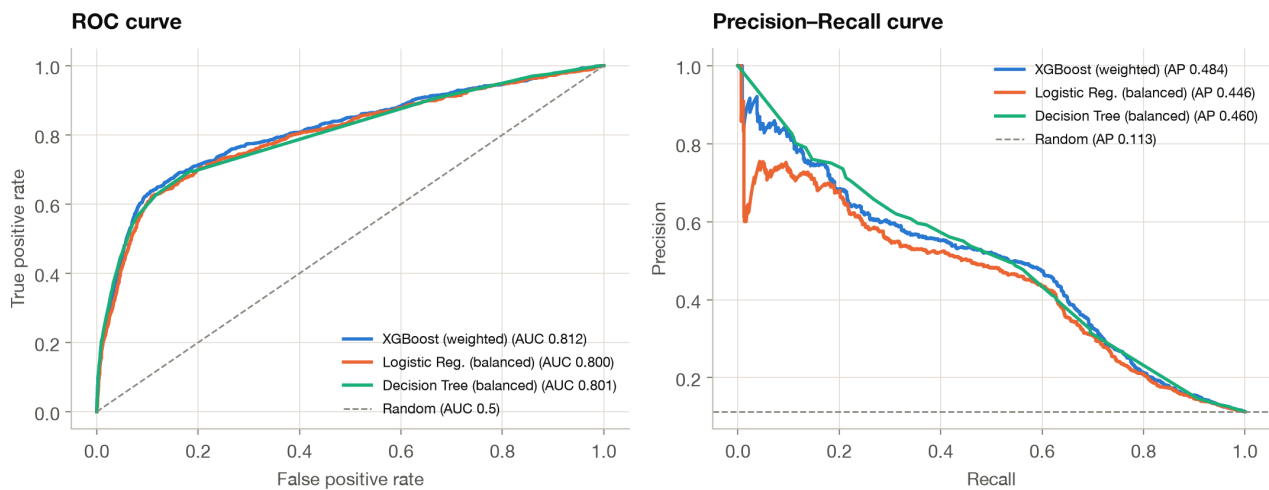


Figure 26. ROC (left) and precision–recall (right) curves for the three leading model families on the test set.

XGBoost (weighted): confusion matrices (test set, n = 8,236)

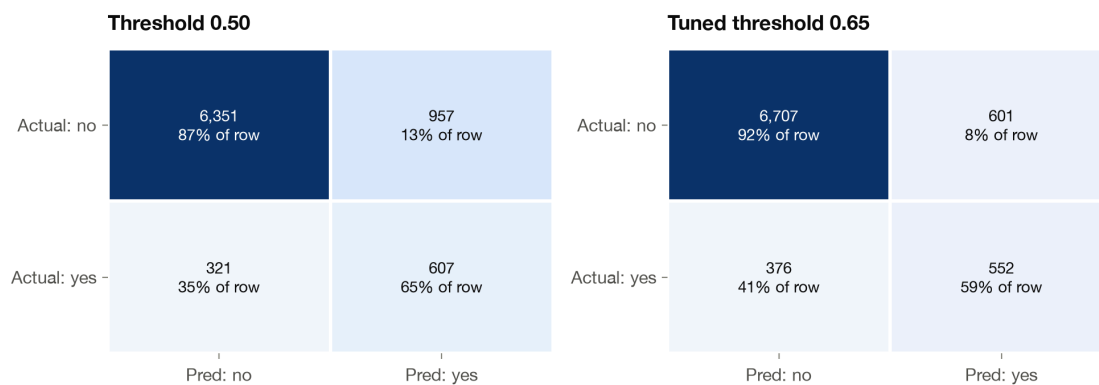


Figure 27. Confusion matrices for class-weighted XGBoost at the default and the tuned threshold.

The threshold is a business lever. Raising the threshold from 0.50 to 0.65 cuts wasted calls (false positives) from 957 to 601 (–37%). Subscribers found fall only from 607 to 552, and precision rises from 0.39 to 0.48. A bank would set this threshold from the cost of a call and the value of a deposit.

6.4 Interpretation of the best model

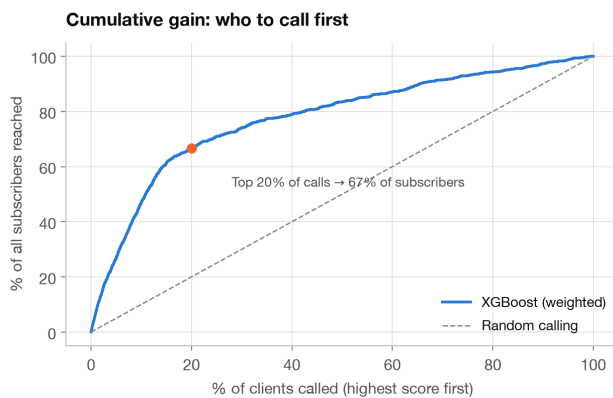


Figure 28. Cumulative gain: share of subscribers reached when calling clients in model-score order.

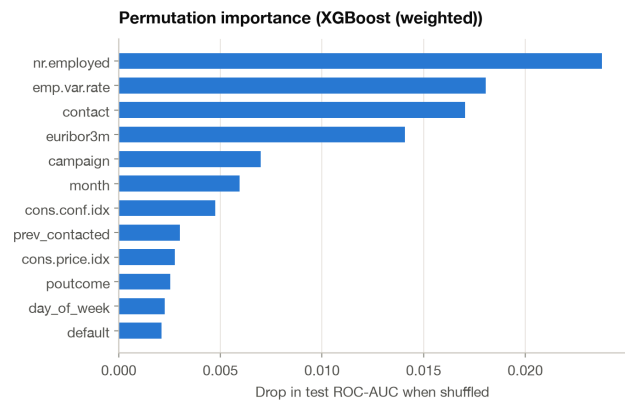


Figure 29. Permutation importance (drop in test ROC-AUC).

Business value. If the bank calls clients in the order the model ranks them, the first 20% of calls reach 67% of all subscribers, a **3.3× lift** over random calling. The same number of deposits could be secured with far fewer calls.

Drivers. The economic context (`nr.employed`, `emp.var.rate`, `euribor3m`) matters most, followed by contact channel (cellular 14.7% vs landline 5.2%), the number of calls in the campaign, and the month. Recommendations:

1. Prioritise clients by model score and stop where the expected deposit value no longer covers the cost of a call.
2. Re-target clients whose previous campaign succeeded (65% conversion).
3. Prefer mobile contact and cap repeated calls to the same client within a campaign.
4. Time campaigns with the economic cycle; conversion is much higher when interest rates and employment are low.

7 Comparative Discussion

Table 4. Summary of the two supervised tasks.

	Regression (CCPP)	Classification (Bank Marketing)
Target	Continuous: energy output (MW)	Binary: subscribed (11.3% yes)
Models compared	10 families + tuned XGBoost	Baseline + 9 models + threshold-tuned XGBoost
Primary metric	RMSE (also MAE, R^2)	F1 and ROC-AUC (also precision, recall, PR-AUC)
Best model	XGBoost (tuned)	XGBoost (class-weighted, threshold 0.65)
Best result	RMSE 3.17 MW, R^2 0.966	F1 0.531, ROC-AUC 0.812, 3.3× lift
Simple baseline	Linear Regression: RMSE 4.59 MW	Plain Logistic Regression: F1 0.320
Key challenge	Non-linearity, multicollinearity	Class imbalance, target leakage
Most important feature	Ambient temperature (AT)	Macro-economy (employment, Euribor)

In both tasks, gradient-boosted trees (XGBoost) gave the best results. They capture non-linear effects and feature interactions without manual feature engineering. The size of the gain depends on the problem. On the physically smooth CCPP data, simple linear models were already strong and boosting added a meaningful 31% reduction in error. On the bank data, the choice of metric and the handling of imbalance mattered far more than the choice of algorithm: every reasonable model reached similar ROC-AUC, but only models that accounted for the 11% positive rate found most subscribers. In both tasks, cross-validation agreed closely with test performance, which gives confidence that the reported results will generalise.

8 Conclusion, Limitations and Future Work

8.1 Conclusion

- **App ecosystem:** installs concentrate in Games and Communication; free apps dominate volume; ratings are inflated and depend more on maintenance than on size or price.
- **Game industry:** a hit-driven, concentrated market that peaked in 2008. North America is half of global sales, and Japan has distinct preferences led by Role-Playing games.
- **Regression:** tuned XGBoost predicts power plant output to within about 3 MW ($R^2 = 0.966$); temperature is the dominant driver.
- **Classification:** a leakage-free, imbalance-aware XGBoost pipeline ranks bank clients well enough to reach two-thirds of subscribers with one-fifth of the calls.

8.2 Limitations

- The Play Store data is a single 2018 snapshot; ratings are self-selected and installs are coarse buckets.
- The video game data counts boxed retail units only, so recent years and digital sales are undercounted.
- The bank data covers 2008–2010, which includes the financial crisis. Relationships with macro-economic indicators may not transfer to other periods, and a random split does not test performance on future campaigns.
- Only XGBoost was tuned systematically; other models use sensible default hyper-parameters.

8.3 Future work (PBL 2)

- Tune all leading models (randomised or Bayesian search) and explain predictions with SHAP values.
- Choose the classification threshold by expected profit (call cost vs deposit value) instead of F1.
- Validate the bank model on later campaigns (time-based split) to measure robustness over time.
- Package the best models as a simple interactive prediction app.

References

1. Gupta, L. (2019). *Google Play Store Apps* [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/lava18/google-play-store-apps>
2. Smith, G. (2016). *Video Game Sales* [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/gregorut/videogamesales>
3. Tüfekci, P. (2014). Prediction of full load electrical power output of a base load operated combined cycle power plant using machine learning methods. *International Journal of Electrical Power & Energy Systems*, 60, 126–140. Dataset: UCI ML Repository #294.
4. Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62, 22–31. Dataset: UCI ML Repository #222.
5. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.

6. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proc. KDD*, 785–794.
7. Chawla, N. V. et al. (2002). SMOTE: Synthetic minority over-sampling technique. *JAIR*, 16, 321–357.

Appendix: Reproducibility

All code is in the project repository under `code/` :

Path	Contents
<code>notebooks/01_eda_play_store.ipynb</code>	Play Store cleaning and EDA (Section 4.1)
<code>notebooks/02_eda_video_game_sales.ipynb</code>	Video game sales EDA (Section 4.2)
<code>notebooks/03_regression_ccpp.ipynb</code>	CCPP regression, tuning and diagnostics (Section 5)
<code>notebooks/04_classification_bank_marketing.ipynb</code>	Bank Marketing classification (Section 6)
<code>src/viz.py</code>	Shared paths and plot styling
<code>data/raw/</code> , <code>data/processed/</code>	Original downloads and cleaned datasets
<code>outputs/figures/</code> , <code>outputs/models/</code>	All figures and result tables used in this report

Environment: `python3.13 -m venv .venv && .venv/bin/pip install -r requirements.txt`, then run the notebooks in order. All random operations use `random_state = 42`.